

MLflow Logging at Scale — Best Practices

Customer: Albertsons · **Databricks contacts:** Akil Thomas, Siddharth Majumdar · **Date:** 2026-06-11

Situation

Albertsons logs many small forecasting models with MLflow — one run per (region × SKU × model type), across three model families (Arima, XGBoost, and a third). At their volume this produces hundreds of thousands of runs per experiment and they are seeing:

- Large latency when logging models/runs
- Apparent rate limits
- Failures in the training pipeline
- The 500,000-runs-per-experiment cap

The key reframe

In every comparable case we've handled, **"rate limits / failures under concurrent logging" was NOT the workspace API rate limit — it was the `mlflowdbfs` artifact-upload path**. Two resolved escalations with nearly identical shapes (hundreds of small models logged concurrently) both root-caused to `mlflowdbfs`, and in one of them rate limiting was explicitly ruled out (the workspace was within the artifact rate limit the whole time). The fix in both was a single environment variable.

This means the first thing to try is not a quota increase — it's disabling `mlflowdbfs`.

Recommendations

1. Disable mlflowdbfs — the #1 lever

Set this before importing MLflow (ideally as a cluster environment variable on the training cluster):

```
import os
os.environ["DISABLE_MLFLOWDBFS"] = "true"
import mlflow
```

This routes artifact uploads through the Python HTTP path with thread-safe credentials, avoiding DBFS-FUSE filesystem overhead and Py4J socket contention. In the comparable cases this took model-logging success from ~48% to 100% and removed most of the latency. Use a recent MLflow client so it includes the fix that skips a wasted ACL-protected write attempt (which otherwise costs 1–6 seconds per model).

Why mlflowdbfs fails under concurrency: its credentials are resolved JVM-side and do not propagate to worker threads in a `ThreadPoolExecutor`, producing 401s and connection-pool

timeouts — exactly the "rate limit / failure" symptom, but it is an authentication/IO problem, not throttling.

2. Disable or scope autolog — especially for XGBoost

`mlflow.xgboost.autolog()` (and `mlflow.autolog()`) logs per-iteration metrics and a model per fit. Across thousands of region×SKU fits this multiplies both per-run API calls (latency) and total run count (filling the 500k cap). For many-small-model pipelines, disable autolog and log explicitly, or use `autolog(log_models=False)`.

3. Batch the writes and decouple training from logging

- **Batch params/metrics:** one `mlflow.log_params(dict)` / `mlflow.log_metrics(dict)` call instead of per-key calls. For Arima/XGBoost with many hyperparameters this collapses dozens of round-trips into one.
- **Async logging:** `mlflow.config.enable_async_logging()` flushes metrics/params/tags on background threads, cutting perceived latency inside the training loop.
- **Bounded uploader pool:** train on one pool, upload models on a separate pool with capped concurrency (~8–16 threads) to avoid exhausting the HTTP connection pool.
- **Tune client retries** for transient blips: `MLFLOW_HTTP_REQUEST_MAX_RETRIES`, `MLFLOW_HTTP_REQUEST_TIMEOUT`.

4. Split experiments — don't just raise the 500k cap

The 500,000 runs/experiment limit can be raised via a quota request, but raising it trades query and storage cost on a shared database shard, and the UI/search slow down as a single experiment grows. The better pattern is to shard experiments along the natural topology.

- Use **one experiment per region**, or **per region × model-family** (Arima / XGBoost / third).
- Runs accumulate on every retrain cycle, not just once — so even a moderate SKU count crosses 500k over months. Sharding keeps each experiment fast and well under the cap.
- Push model storage to the **Unity Catalog** registry (`databricks-uc`) rather than the workspace DBFS artifact store, for better scaling and governance.
- Measure current state with the `system.mlflow.runs_latest` system table to see actual runs-per-experiment and size the split.

5. If genuine 429s appear

Then it is the real limits: the workspace MLflow tracking API (~120 RPS/workspace) and the artifact upload limit (~35 req/s). Request an increase through the standard quota process — but only after disabling `mlflowdbfs`, turning off autolog, and batching, because those usually remove the apparent "rate limit" entirely.

Diagnostic checklist

Have the customer capture the actual error code — it tells you which problem it is:

Symptom	Likely cause	Action
401 Unauthorized under concurrency	mlflowdbfs credentials not propagating to worker threads	<code>DISABLE_MLFLOWDBFS=true</code>
Connection pool timeouts / slow uploads	DBFS-FUSE IO + Py4J contention	<code>DISABLE_MLFLOWDBFS=true</code> + bounded uploader pool
429 Too Many Requests	Genuine API/artifact rate limit	Batch + async first; quota increase as last resort
Experiment slow / approaching 500k runs	Too many runs in one experiment	Split by region × model-family; disable autolog

Priority order

1. `DISABLE_MLFLOWDBFS=true` (cluster env var)
2. Disable/scope autolog
3. Batch `log_params/log_metrics` + enable async logging + bounded uploader pool
4. Split experiments by region × model-family; log models to Unity Catalog
5. Quota increase only if real 429s persist after the above